GPyTorch SVGP dtype errors and FreeSurfer pipeline fixes

The "Unsupported dtype for variational_cholesky_jitter" error fires when GPyTorch's variational machinery encounters any dtype other than `float32`, `float64`, or `float16` — most often caused by mixing NumPy-derived float64 data with GPyTorch's default float32 parameters, or by custom spectral kernels whose eigendecomposition silently upcasts to float64. Every component of an SVGP model — inducing points, training data, kernel parameters, variational distribution, and likelihood — must share a single dtype. The fix is straightforward: cast at the earliest pipeline stage and verify before training. Below is a detailed breakdown covering all five topics.

How `variational_cholesky_jitter` triggers the RuntimeError

The error originates in GPyTorch's `_dtype_value_context` base class in `settings.py`. The class maintains separate jitter values for `torch.float`, `torch.double`, and `torch.half`, and raises `RuntimeError(f"Unsupported dtype for {cls.__name__}.")` for anything else. Gpytorch Gpytorch The `variational_cholesky_jitter` subclass sets **1e-4 for float32** and **1e-6 for float64** Gpytorch Gpytorch — more jitter at lower precision to compensate for reduced numerical headroom.

Three scenarios commonly trigger the error. First, `torch.bfloat16` tensors reaching the jitter lookup (e.g., through `torch.autocast`). Second, a `None` dtype propagating when a kernel's output tensor lacks an explicit dtype. Third — and most relevant to spectral kernels — internal linear algebra operations returning float64 tensors that flow into a float32 variational strategy. GPyTorch's `linalg_dtypes` setting defaults to performing Cholesky and eigendecompositions in float64 even when the model is float32, Gpytorch and if the resulting float64 output isn't cast back before reaching `variational_cholesky_jitter.value(dtype)`, the dtype lookup fails.

The context manager accepts explicit overrides, which is useful when encountering `NotPSDError` after jitter escalation:

```python
with gpytorch.settings.variational_cholesky_jitter(float_value=1e-3, double_
    output = model(train_x)
```

Relevant GitHub issues include **#648** (float64 data with float32 RBF kernel), github **#1452** ("Found dtype Float but expected Double" in `_matmul`), github GitHub **#1225** (`torch.from_numpy()` producing float64 causing lazy tensor conflicts), GitHub and **#1457** (inconsistent jitter application in `psd_safe_cholesky`). GitHub

Every SVGP component must share one dtype

Both float32 and float64 work in GPyTorch SVGP models, but **all six components must match exactly**: inducing points, training inputs, training targets, kernel hyperparameters, variational distribution parameters (mean and Cholesky factor), and likelihood parameters. The inducing points are particularly critical because `CholeskyVariationalDistribution` derives its parameter dtype from them Gpytorch — if inducing points are float64 but the kernel is float32, the mismatch cascades through the variational strategy.

Component	Dtype source	Common pitfall
Inducing points	Explicit at creation	`torch.randn()` defaults to float32; numpy-derived points are float64 codestudy
Training data X, y	Explicit at creation	`torch.from_numpy()` preserves numpy's float64 GitHub
Kernel parameters	Model default dtype	Created as float32 unless `model.double()` called PyTorch
Variational params	Inherited from inducing points	Silently mismatches if inducing points differ from kernel
Likelihood params	Likelihood default dtype	Must call `likelihood.to(dtype=...)` separately

A reliable verification pattern before training:

```python
DTYPE = torch.float32
for name, param in model.named_parameters():
    assert param.dtype == DTYPE, f"{name}: {param.dtype}"
assert train_x.dtype == DTYPE and train_y.dtype == DTYPE
```

Casting NumPy float64 from nibabel into float32 models

The single most common source of GPyTorch dtype errors in neuroimaging pipelines is nibabel's `get_fdata()`, which **returns float64 by default** GitHub codestudy (per BIAP-8, to prevent silent integer overflow). Both `torch.tensor(numpy_array)` and `torch.from_numpy(numpy_array)` preserve this float64, GitHub GitHub creating an invisible mismatch with GPyTorch's float32 parameters. codestudy

The fix is to cast at the earliest possible stage. Since nibabel 3.0, `get_fdata()` accepts an explicit dtype argument:

```python
import nibabel as nib
import numpy as np
import torch

DTYPE = torch.float32

# Cast at load time — never creates a float64 intermediate
data = nib.load('lh.thickness.fwhm10.fsaverage.mgh').get_fdata(dtype=np.floa

# FreeSurfer geometry returns float64; cast immediately
coords, faces = nib.freesurfer.read_geometry('lh.white')
coords = coords.astype(np.float32)

# Morph data (curv files) — may be big-endian float
thickness = np.asarray(nib.freesurfer.read_morph_data('lh.thickness'), dtype

# Tensor conversion with explicit dtype
train_x = torch.tensor(coords, dtype=DTYPE)
train_y = torch.tensor(thickness, dtype=DTYPE)
inducing_points = train_x[np.random.choice(len(train_x), 500, replace=False)

# Model and likelihood must also match
model = SVGPModel(inducing_points).to(dtype=DTYPE)
likelihood = gpytorch.likelihoods.GaussianLikelihood().to(dtype=DTYPE)
```

For SVGP models on large cortical surfaces (~160k vertices), float32 is recommended for memory efficiency. Runebook.dev For small-data Exact GPs, BoTorch's developers recommend float64 because single-precision arithmetic more frequently produces singular covariance matrices. GitHub GitHub The `model.double()` and `model.float()` methods convert all parameters, buffers, inducing points, and variational parameters in a single call.

`mri_surf2surf` does write .nii.gz, but MGH is safer

FreeSurfer's `mri_surf2surf` **supports any format that** `mri_convert` **supports**, including `.nii.gz`. Freesurfer The output format is determined by the file extension or the `--trg_type` flag. CRAN Bruce Fischl confirmed on the mailing list: "if you specify .nii or .nii.gz as the extension of any output file we will write it in nifti format." The Mail Archive

However, there is a critical gotcha. **NIfTI v1 limits each dimension to 32,767**, and fsaverage hemispheres have ~163,842 vertices. Neuroconductor When NIfTI output is detected, FreeSurfer automatically reshapes the data — for example, storing 163,842 vertices as 27,307 × 1 × 6 rather than 163,842 × 1 × 1. Freesurfer GitHub This "FreeSurfer NIfTI hack" means standard NIfTI readers (nibabel, FSL) see a pseudo-3D volume, and you must flatten the array to recover the original vertex ordering. Free Surfer

The affine matrix in a surface-overlay NIfTI file **has no spatial meaning**. Surface overlays are inherently 1D — one scalar per vertex — and the vertex-to-coordinate mapping lives in the surface mesh file (lh.white, rh.pial), not in any volume header. Nben FreeSurfer writes the vox2ras from the conformed volume geometry (256 × 256 × 256, 1mm isotropic) into both sform and qform, Nih but this is an artifact of storing non-volumetric data in a volumetric container. Ignore the affine entirely for surface data.

MGH/MGZ is strongly preferred for surface overlays because there is no 32k dimension limit, no reshaping is needed, and the data shape is a clean `(nvertices, 1, 1, nframes)`. Free Surfer The recommended workflow is to keep data in MGH/MGZ throughout FreeSurfer processing and convert to NIfTI only for export to non-FreeSurfer tools. Nih

The "Unknown MGH format version" error is almost always file corruption

Nibabel's `MGHHeader.chk_version()` reads the first 4 bytes of an MGH file as a big-endian int32 and checks that it equals **1** — the only version in the MGH format specification. GitHub **No version of FreeSurfer (5.x through 8.x) or FastSurfer has ever written a different version number.** The format has been stable since its inception; FastSurfer itself uses nibabel internally for MGH I/O, Internet Archive so its outputs conform by construction.

The three real causes of this error are:

- **Opening .mgz files with raw `open()` instead of `nib.load()`** — since MGZ is gzip-compressed, raw binary reading yields gzip headers in the first 4 bytes, which don't decode as version 1. Nben This is the most commonly reported cause (documented in a 2019 neuroimaging mailing list thread). Python
- **Corrupted or truncated files** — partial downloads, disk errors, or interrupted writes.
- **Misidentified file types** — files with `.mgh`/`.mgz` extensions that aren't actually MGH format (e.g., a NIfTI file renamed with an `.mgz` extension).

There is **no nibabel version that "fixes" this** because there is nothing to fix — the check is correct. When nibabel loads with `fix=True` (the default), it silently corrects the version to 1 and continues, GitHub so under normal loading conditions the error manifests as a warning rather than a crash. The only relevant recent MGH change in nibabel is **v5.4.0** (March 2026), which added `ushort` datatype support for MGH files. nipy

`mri_convert` is not a standard workaround for this error; `nib.load()` handles MGZ decompression automatically. GitHub +2 If a file genuinely fails to load, the issue is file integrity, not format version incompatibility.

Spectral kernels and the float64 eigendecomposition trap

GPyTorch's `SpectralMaternKernel` **is not a built-in kernel** — it would be a custom or third-party implementation. The built-in `SpectralMixtureKernel` exists but does not perform eigendecomposition. GitHub Any custom spectral kernel that calls `torch.linalg.eigh()` faces a specific dtype hazard: GPyTorch's `linalg_dtypes` setting defaults to `torch.float64` for eigendecompositions, Gpytorch Gpytorch and PyTorch itself may upcast float32 inputs to float64 internally for numerical stability PyTorch (documented in PyTorch's numerical accuracy notes, and in PyTorch issue **#105359** where `eigh` fails on GPU with float32 but succeeds with float64). GitHub

When float64 eigenvalues and eigenvectors flow back into a float32 variational strategy, two failure modes occur. The benign case is a "expected dtype Float but found Double" error in a matmul. GitHub The more confusing case is the "Unsupported dtype" error if an unexpected intermediate dtype reaches `variational_cholesky_jitter`.

The fix for custom spectral kernels is explicit dtype management after every eigendecomposition:

```python
class SafeSpectralKernel(gpytorch.kernels.Kernel):
    def forward(self, x1, x2, **params):
        K = self._compute_spectral_matrix(x1, x2)
        eigenvalues, eigenvectors = torch.linalg.eigh(K)
        # Cast back to input dtype immediately
        eigenvalues = eigenvalues.to(x1.dtype)
        eigenvectors = eigenvectors.to(x1.dtype)
        return self._reconstruct(eigenvalues, eigenvectors)
```

Alternatively, disable the automatic upcasting (at the cost of numerical stability):

```python
with gpytorch.settings.linalg_dtypes(default=torch.float32):
    output = model(train_x)
```

Conclusion

The dtype consistency requirement in GPyTorch SVGP models is strict but manageable: pick float32 or float64 and enforce it everywhere, from nibabel loading (`get_fdata(dtype=np.float32)`) through tensor creation (`torch.tensor(..., dtype=DTYPE)`) codestudy to model initialization (`model.to(dtype=DTYPE)`). The `variational_cholesky_jitter` error is a symptom of dtype leakage, not a root cause — trace upstream to find where float64 entered a float32 pipeline. For neuroimaging workflows, keep surface data in MGH/MGZ throughout processing to avoid NIfTI's 32k reshape complications. Nih The "Unknown MGH format version" error is a file-integrity issue, not a tool-version incompatibility — verify that `.mgz` files are being opened with `nib.load()` rather than raw file handles. Custom spectral kernels must explicitly cast eigendecomposition outputs back to the model dtype, because both GPyTorch and PyTorch silently upcast linear algebra operations to float64 for stability. Gpytorch